

REACT PROJECT

Bookmark Manager

Components • Props • State • Forms • Lists • useEffect • Composition

პროექტის ტიპი: Individual React Application

მთავარი მიზანი: ნასწავლი React თემების გაერთიანება ერთ რეალურ, ინტერაქტიულ აპლიკაციაში.

Figma Design: <https://www.figma.com/design/giO3ChUIk9nSfgzW6aEVh5/bookmark-manager-app?node-id=234-4992&m=dev&t=eztu8u7hiTorkICO-1>

Project Brief

თქვენი დავალებაა შექმნათ Bookmark Manager — React აპლიკაცია, რომელიც მომხმარებელს საშუალებას მისცემს შეინახოს, მოძებნოს, გაფილტროს და მართოს მისთვის საინტერესო ვებგვერდების ბმულები.

პროექტი უნდა იყოს სრულად ინტერაქტიული და არა მხოლოდ დიზაინის ვიზუალური კოპირება. მთავარი ყურადღება ექცევა React-ის სწორ გამოყენებას, state-ის მართვას, კომპონენტების სტრუქტურას და მომხმარებლისთვის სხვადასხვა UI მდგომარეობის სწორად ჩვენებას.

მნიშვნელოვანი: დიზაინის ზუსტი ვიზუალური გამეორება მნიშვნელოვანია, მაგრამ ფუნქციონალი და React-ის სწორი არქიტექტურა უფრო მაღალი პრიორიტეტია.

აპლიკაციაში მომხმარებელს უნდა შეეძლოს

- ახალი Bookmark-ის დამატება;
- არსებული Bookmark-ის რედაქტირება;
- Bookmark-ის წაშლა;
- Bookmark-ის დაარქივება და არქივიდან დაბრუნება;
- Bookmark-ის Pin / Unpin;
- Bookmark-ების ძებნა;
- Tag-ებით გაფილტვრა;
- Bookmark-ების დალაგება;

- Light / Dark Theme-ის შეცვლა;
- მონაცემების შენარჩუნება გვერდის გადატვირთვის შემდეგ.

Project Goal

პროექტის მთავარი მიზანია React-ში უკვე ნასწავლი თემების ერთ რეალურ აპლიკაციაში გაერთიანება. შეფასებისას მნიშვნელოვანია არა მხოლოდ ის, რომ ფუნქცია მუშაობდეს, არამედ ისიც, თუ რამდენად სწორად არის გადაწყვეტილი მისი React არქიტექტურა.

- როგორ არის დაყოფილი UI კომპონენტებად;
- სად და რატომ ინახავთ state-ს;
- როგორ გადაეცემა მონაცემები კომპონენტებს შორის;
- როგორ განაახლებთ array/object state-ს;
- როგორ მართავთ forms-ს და validation-ს;
- როგორ იყენებთ useEffect-ს;
- როგორ ამუშავებთ loading, empty და error states-ს;
- რამდენად reusable არის კომპონენტები;
- რამდენად გასაგებია პროექტის ფაილების და კოდის სტრუქტურა.

Data Source & Project Structure

საწყისი მონაცემები შეინახეთ `public/data/bookmarks.json` ფაილში. ეს ფაილი წარმოადგენს seed data-ს — ანუ საწყის მონაცემებს, რომლებიც გამოიყენება მხოლოდ იმ შემთხვევაში, თუ მომხმარებელს `localStorage`-ში ჯერ არ აქვს შენახული Bookmark-ები.

```
bookmark-manager/  
|  
|—— public/  
|   |—— data/  
|       |—— bookmarks.json  
|  
|—— src/  
|   |—— components/  
|   |—— App.jsx  
|   |—— main.jsx  
|  
|—— package.json  
|—— ...
```

`bookmarks.json` არ უნდა დააიმპორტოთ პირდაპირ JavaScript module-ად. მონაცემები უნდა ჩაიტვირთოს HTTP request-ის სიმულაციით — `fetch()`-ის საშუალებით.

```
fetch("/data/bookmarks.json")
```

Bookmark Data Structure

მზა მონაცემები წარმოდგენილია ცალკე ფაილად დირექტორიაში

https://drive.google.com/drive/folders/1Vefm1LDIQecRTE2brAc8LeZmRUZUb_Vu?usp=sharing

Functional Requirements

1. Bookmark List

- აპლიკაციის მთავარ გვერდზე უნდა გამოჩნდეს Bookmark-ების სია.
- თითოეულ Bookmark Card-ზე მინიმუმ უნდა ჩანდეს: Title, URL ან domain, Description, Tags, Pin/Unpin, Edit, Archive და Delete actions.
- Bookmark-ების სია უნდა შეიქმნას array-ის .map() მეთოდით.
- თითოეულ list item-ს უნდა ჰქონდეს უნიკალური და სტაბილური key. Array index არ გამოიყენოთ key-დ, თუ Bookmark-ს საკუთარი id აქვს.

2. Add Bookmark

- მომხმარებელს უნდა შეეძლოს ახალი Bookmark-ის დამატება.
- ფორმა მინიმუმ უნდა შეიცავდეს: Title, URL, Description და Tags.
- ფორმა უნდა იყოს controlled form — input-ების მნიშვნელობები იმართებოდეს React state-ით.

```
<input  
  value={form.title}  
  onChange={handleChange}  
>
```

3. Form Validation

- Title სავალდებულოა და ცარიელი მნიშვნელობა არ უნდა მიიღოს.
- URL სავალდებულოა და უნდა ჰქონდეს სწორი URL ფორმატი.
- Description შეიძლება იყოს optional.
- Bookmark-ს უნდა ჰქონდეს მინიმუმ ერთი Tag.
- თუ validation ვერ გაიარა, Bookmark არ უნდა დაემატოს და შესაბამის field-თან უნდა გამოჩნდეს error message.

Title is required
Please enter a valid URL

4. Edit Bookmark

- მომხმარებელს უნდა შეეძლოს არსებული Bookmark-ის რედაქტირება.
- Edit-ისას ფორმაში წინასწარ უნდა ჩაიტვირთოს არჩეული Bookmark-ის მიმდინარე მონაცემები.
- Save-ის შემდეგ არსებული Bookmark უნდა განახლდეს და ახალი არ უნდა შეიქმნას.
- Array/object state-ის პირდაპირი mutation დაუშვებელია.

5. Delete Bookmark

- მომხმარებელს უნდა შეეძლოს Bookmark-ის წაშლა.
- წაშლისას შესაბამისი Bookmark უნდა გაქრეს state-იდან და UI-დან.
- სასურველია Delete-ის წინ confirmation გამოჩნდეს.

6. Pin / Unpin

- თითოეული Bookmark-ის Pin / Unpin შესაძლებელი უნდა იყოს.
- Pinned Bookmark-ები ჩვეულებრივ Bookmark-ებზე წინ უნდა გამოჩნდნენ.
- UI-ში ვიზუალურად უნდა ჩანდეს pinned მდგომარეობა.

7. Archive

- მომხმარებელს უნდა შეეძლოს Bookmark-ის დაარქივება.
- აპლიკაციას უნდა ჰქონდეს მინიმუმ ორი View: All Bookmarks და Archived.
- All Bookmarks-ში archived Bookmark-ები არ უნდა გამოჩნდეს.
- Archived view-ში უნდა გამოჩნდეს მხოლოდ archived Bookmark-ები.
- Archived Bookmark-ის Restore შესაძლებელი უნდა იყოს.

8. Search

- აპლიკაციას უნდა ჰქონდეს Search input.
- ძებნა მინიმუმ უნდა მუშაობდეს Title-ისა და Description-ის მიხედვით.
- Search result უნდა განახლდეს input-ის ცვლილებისას.
- თუ არცერთი Bookmark არ დაემთხვა ძებნას, უნდა გამოჩნდეს Empty State.

9. Filtering by Tags

- მომხმარებელს უნდა შეეძლოს Bookmark-ების გაფილტვრა Tag-ის მიხედვით.
- უნდა არსებობდეს All option.
- არჩეული Tag-ის მიხედვით UI-ში მხოლოდ შესაბამისი Bookmark-ები უნდა გამოჩნდეს.

10. Sorting

- Bookmark-ებს უნდა ჰქონდეთ მინიმუმ ორი Sort ვარიანტი, მაგალითად Newest/Oldest ან A-Z/Z-A.
- Sort-ის შეცვლისას Bookmark list ავტომატურად უნდა განახლდეს.

11. Empty States

- აუცილებლად უნდა არსებობდეს Empty State-ები: საერთოდ ცარიელი list, Search-ის ცარიელი შედეგი და ცარიელი Archived view.
- არ დატოვოთ უბრალოდ ცარიელი ეკრანი.

12. Loading State

- აპლიკაციის პირველ ჩატვირთვაზე უნდა არსებობდეს Loading State.
- საწყისი მონაცემების ჩატვირთვა მოახდინეთ ასინქრონული პროცესის მსგავსად, მაგალითად setTimeout()-ით.
- Loading-ის სამართავად გამოიყენეთ state და useEffect.

13. localStorage

- Bookmark-ების მდგომარეობა გვერდის refresh-ის შემდეგ არ უნდა დაიკარგოს.
- localStorage-ში მინიმუმ შეინახეთ bookmarks და theme.
- Refresh-ის შემდეგ აპლიკაციამ უნდა აღადგინოს წინა მდგომარეობა.

14. Light / Dark Theme

- აპლიკაციას უნდა ჰქონდეს Light Mode და Dark Mode.
- Theme-ის შეცვლა უნდა მოხდეს state-ით.
- არჩეული Theme უნდა შეინახოს localStorage-ში და refresh-ის შემდეგ აღდგეს.

React Technical Requirements

პროექტში სავალდებულოდ უნდა გამოიყენოთ შემდეგი React კონცეფციები და patterns:

- Functional Components
- JSX
- Props და Props destructuring
- props.children
- Reusable Components
- Callback Props
- useState
- Event Handlers
- Controlled Inputs
- onChange
- onSubmit და preventDefault
- Array .map()
- Stable key
- Immutable array/object updates
- Conditional Rendering
- useEffect
- Dependency Array

- localStorage
- Loading State
- Empty State
- Component Composition

Component Requirements

აპლიკაცია უნდა დაიყოს ლოგიკურ reusable components-ად. ქვემოთ მოცემული სტრუქტურა მხოლოდ მაგალითია — ზუსტად გამეორება სავალდებულო არ არის.

```
App
├── Layout
│   ├── Header
│   └── Sidebar
├── SearchBar
├── FilterBar
├── SortSelect
├── BookmarkList
│   ├── BookmarkCard
│   └── TagList
├── Modal
└── BookmarkForm
```

მოთხოვნა: თქვენ თავად უნდა განსაზღვროთ component boundaries. App.jsx არ უნდა შეიცავდეს მთელ აპლიკაციის markup-სა და ყველა პასუხისმგებლობას.

Component Composition

პროექტში მინიმუმ ერთი კომპონენტი უნდა იყენებდეს props.children-ს. მაგალითად:

```
<Modal>
  <BookmarkForm />
</Modal>
```

Wrapper component უნდა იყოს reusable და არ უნდა იყოს მიზნული კონკრეტულ Bookmark data-ზე.

Props & Callback Props

```
<BookmarkCard
  bookmark={bookmark}
  onEdit={handleEdit}
  onDelete={handleDelete}
/>
```

Child component-მა მიღებული props პირდაპირ არ უნდა შეცვალოს. Parent state-ის ცვლილების ინიცირებისთვის გამოიყენეთ callback props.

State Management

- State უნდა იყოს იმ კომპონენტში, რომელსაც შესაბამისი მონაცემების მართვა ეკუთვნის.
- არ შექმნათ ერთი დიდი state მხოლოდ იმიტომ, რომ ყველა მონაცემი ერთ ადგილას იყოს.
- არ შეინახოთ state-ში მნიშვნელობა, რომელიც პირდაპირ შეიძლება გამოითვალოს უკვე არსებული state-იდან.
- მაგალითად, bookmarks + searchTerm + selectedTag-იდან filtered bookmarks ხშირად render-ის დროს შეიძლება გამოითვალოს.

Conditional Rendering

Conditional Rendering გამოიყენეთ მინიმუმ შემდეგ შემთხვევებში:

- Loading
- Empty list
- Search result not found
- Modal open / closed
- Form validation errors
- Archived / active states
- Pin status
- Light / Dark Theme-ის შესაბამისი UI
- Confirmation UI, თუ გამოიყენებთ

```
condition && <Component />
condition ? <A /> : <B />
```

useEffect Requirements

- Initial Data Loading — პირველი render-ის შემდეგ ჩაიტვირთოს საწყისი Bookmark data.
- localStorage Synchronization — Bookmark-ების ცვლილებებისა მონაცემები შეინახოს localStorage-ში.
- Theme Synchronization — არჩეული Light/Dark Theme შენარჩუნდეს refresh-ის შემდეგ.
- სადაც საჭიროა cleanup, გამოიყენეთ cleanup function.
- Dependency array უნდა იყოს გააზრებულად შერჩეული და effect არ უნდა ქმნიდეს infinite loop-ს.

აკრძალვები

React-ის გარეშე შესრულება — Plain HTML/CSS/JavaScript ვერსია არ ჩაითვლება შესრულებულ პროექტად.

ერთი დიდი App.jsx — მთელი UI და ყველა handler ერთ კომპონენტში არ უნდა იყოს თავმოყრილი.

Props-ის შეცვლა — არ შეიძლება props.title = "New title" ან სხვა პირდაპირი ცვლილება.

State-ის პირდაპირი Mutation — არ გამოიყენოთ bookmarks.push(...), bookmarks[index].title = ... ან form.title =

Array Index როგორც Key — თუ Bookmark-ს აქვს უნიკალური id, key={index} არ გამოიყენოთ.

DOM Manipulation — UI-ის სამართავად არ გამოიყენოთ `document.querySelector()`, `innerHTML`, `style.display` ან `classList`.

Reload მონაცემების განახლებისთვის — Bookmark-ის `add/edit/delete` შემდეგ არ გამოიყენოთ `window.location.reload()`.

Hardcoded Bookmark Cards — Bookmark-ები უნდა `render`-დებოდეს `array data`-დან `.map()`-ით.

Uncontrolled Main Form — ძირითადი `Add/Edit form` უნდა იყოს `controlled`.

External State Management Libraries — არ გამოიყენოთ `Redux`, `Zustand`, `MobX`, `Recoil` და სხვა მსგავსი `library`.

Form Libraries — არ გამოიყენოთ `React Hook Form`, `Formik`, `Yup` ან `Zod` ძირითადი `form/validation`-ისთვის.

შპა Solution-ის კოპირება — დიზაინის გამოყენება შეგიძლიათ, მაგრამ `React logic` უნდა დაწეროთ თავად.

Bonus Tasks

Bonus Tasks სავალდებულო არ არის. მათი მიზანია პროექტის უფრო რთულ დონეზე განვითარება.

- 1. Favorite Bookmark** — Favorite-ად მონიშვნა და მხოლოდ Favorite Bookmark-ების ნახვა.
- 2. Multiple Tag Filtering** — ერთდროულად რამდენიმე Tag-ის არჩევა და შესაბამისი filtering.
- 3. Custom Confirm Modal** — `window.confirm()`-ის ნაცვლად `reusable ConfirmModal`.
- 4. Live Clock / Last Visit** — Header-ში მიმდინარე დრო ან ბოლო ცვლილების დრო; `setInterval`-ის გამოყენებისას `cleanup` აუცილებელია.
- 5. Dynamic document.title** — `useEffect`-ით `browser tab title`, მაგალითად `Bookmarks (12)`.
- 6. Responsive Sidebar** — `Desktop Sidebar`; `Mobile Drawer/Modal interface`.

Acceptance Criteria

პროექტი დასრულებულად ჩაითვლება მხოლოდ მაშინ, თუ ძირითადი მოთხოვნები შესრულებულია.

Application

- ☐ პროექტი მუშაობს `React`-ზე;
- ☐ აპლიკაცია იტვირთება `error`-ის გარეშე;
- ☐ `Console`-ში არ არის მნიშვნელოვანი `React error`;
- ☐ `UI usable` არის `desktop` და `mobile` ზომებზე.

Components

- ☐ `UI` დაყოფილია რამდენიმე ლოგიკურ `component`-ად;

- ☐ არსებობს reusable component-ები;
- ☐ App.jsx არ შეიცავს მთელ UI-ს;
- ☐ გამოყენებულია props და callback props;
- ☐ გამოყენებულია მინიმუმ ერთი composition / props.children pattern.

Bookmark List

- ☐ Bookmark-ები ინახება array-ში;
- ☐ სია render-დება .map()-ით;
- ☐ თითოეულ Bookmark-ს აქვს unique/stable key;
- ☐ Bookmark Card ცალკე component-ია.

Add Bookmark

- ☐ მომხმარებელს შეუძლია Bookmark-ის დამატება;
- ☐ Form controlled-ია;
- ☐ Submit დამუშავებულია onSubmit-ით;
- ☐ გამოყენებულია preventDefault();
- ☐ ახალი Bookmark UI-ში ჩნდება reload-ის გარეშე.

Edit Bookmark

- ☐ Bookmark-ის Edit შესაძლებელია;
- ☐ Edit form წინასწარ აჩვენებს არსებულ მონაცემებს;
- ☐ ცვლილების შემდეგ იგივე Bookmark ახლდება;
- ☐ State პირდაპირ არ იცვლება.

Delete Bookmark

- ☐ Bookmark-ის წაშლა შესაძლებელია;
- ☐ Bookmark state-იდან ქრება;
- ☐ UI reload-ის გარეშე ახლდება.

Validation

- ☐ Title required-ია;
- ☐ URL required-ია;

- ☐ Invalid URL არ მიიღება;
- ☐ მინიმუმ ერთი Tag required-ია;
- ☐ Error messages ჩანს;
- ☐ Invalid form არ submit-დება.

Search

- ☐ Search მუშაობს Title-ზე;
- ☐ Search მუშაობს Description-ზე;
- ☐ შედეგი რეალურ დროში ახლდება;
- ☐ ცარიელ შედეგზე Empty State ჩანს.

Filters & Sorting

- ☐ Tag filtering მუშაობს;
- ☐ არსებობს All option;
- ☐ არსებობს მინიმუმ ორი sort option;
- ☐ Sort-ის შეცვლა სიას სწორად ცვლის.

Archive & Pin

- ☐ Archive შესაძლებელია;
- ☐ Archived Bookmark მთავარ სიაში აღარ ჩანს;
- ☐ არსებობს Archived view და Restore;
- ☐ Pin/Unpin მუშაობს;
- ☐ Pinned Bookmark-ები სიის დასაწყისში ჩანს.

Loading & Empty States

- ☐ პირველ ჩატვირთვაზე Loading State არსებობს;
- ☐ Loading conditional rendering-ით იმართება;
- ☐ initial loading useEffect-თან არის დაკავშირებული;
- ☐ არსებობს empty list, empty search და empty archive states.

useEffect & localStorage

- ☐ useEffect გამოყენებულია რეალური დანიშნულებით;

- ☐ dependency array გააზრებულად არის გამოყენებული;
- ☐ Effect არ ქმნის infinite loop-ს;
- ☐ საჭიროების შემთხვევაში cleanup არსებობს;
- ☐ Bookmark-ები და Theme ინახება localStorage-ში;
- ☐ Refresh-ის შემდეგ მონაცემები და Theme აღდგება.

Code Quality

- ☐ Props პირდაპირ არ იცვლება;
- ☐ State პირდაპირ არ იცვლება;
- ☐ Array update-ები immutable არის;
- ☐ Component names არის PascalCase;
- ☐ Event handler-ებს აქვთ გასაგები სახელები;
- ☐ გაუმართლებელი DOM manipulation არ გამოიყენება;
- ☐ არ არის აშკარა copy-paste duplication;
- ☐ ფაილების სტრუქტურა გასაგებია.

ჩაბარების ფორმატი

პროექტის ჩაბარებისას უნდა წარმოადგინოთ:

1. დასრულებული React პროექტი;
2. GitHub repository;
3. Working deployed version;
4. README.md, სადაც აღწერთ: რას აკეთებს პროექტი, რა ფუნქციები აქვს, რა ტექნოლოგიები გამოიყენეთ და როგორ გაეშვას პროექტი ლოკალურად.

Final Goal: მთავარი მიზანი არ არის მხოლოდ დიზაინის ვიზუალურად გამეორება. მთავარი მიზანია აჩვენოთ, რომ შეგიძლიათ React-ის გამოყენებით ააწყოთ ინტერაქტიული აპლიკაცია, სწორად მართოთ state, components, forms, lists, effects და UI-ის სხვადასხვა მდგომარეობები.